

Applied Deep Learning

Chapter 2: Feedforward Neural Networks

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Fully-Connected FNNs: *Training*

- + *So, what do we do with forward pass?*
- Say the **FNN is fixed** and all **the weights and biases** are given. Forward pass determines the **label** of a given **data-point x** .
- + *Well! But we need to train the network! Right?!*
- Yes! We define the **loss** and then **train** it via **gradient descent**
- + *Then, we need to determine the **gradient**! It sounds complicated!*
- Well! there is an efficient algorithm for that called **backpropagation**

Let's see what **backpropagation** is!

Fully-Connected FNNs: *Training*

Let's recall how we train the network: in our FNN, we considered M outputs; thus, we could assume that the dataset is of the form

$$\mathbb{D} = \{(\mathbf{x}_b, \mathbf{v}_b) \text{ for } b = 1, \dots, B\}$$

Here, we have denoted the **true labels** by $\mathbf{v}_b = [v_{b,1}, \dots, v_{b,M}]^T$ to avoid confusion with the FNN's outputs. Now, let's denote the forward pass by $\mathbf{y}_b = \text{PassF}(\mathbf{x}_b | \mathbf{w})$ with $\mathbf{w}$ is a vector collecting $\{\mathbf{W}_\ell\}$ for $\ell = 1, \dots, L + 1$

Given **data-point** $\mathbf{x}_b$, by forward pass we get $\mathbf{y}_b$ at the output layer of the FNN with weights $\mathbf{w}$. This output is desired to be the **true label** $\mathbf{v}_b$

How do we do the training?

$$\mathbf{w}^\star = \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b) \quad (\text{Training})$$

Fully-Connected FNNs: *Training*

Let's recall how we train the network

$$\begin{aligned}\mathbf{w}^{\star} &= \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) = \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b) \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\operatorname{PassF}(\mathbf{x}_b | \mathbf{w}), \mathbf{v}_b)\end{aligned}$$

- 1: Initiate at some $\mathbf{w}^{(0)} \in \mathbb{R}^D$ and deviation $\Delta = +\infty$
- 2: Choose some small ϵ and η , and set $t = 1$
- 3: **while** $\Delta > \epsilon$ **do**
- 4: Update weights as $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \nabla \hat{R}(\mathbf{w}^{(t-1)})$
- 5: Update the deviation $\Delta = |\hat{R}(\mathbf{w}^{(t)}) - \hat{R}(\mathbf{w}^{(t-1)})|$
- 6: **end while**

In this algorithm, the main challenge is to calculate $\nabla \hat{R}(\mathbf{w}^{(t-1)})$

Fully-Connected FNNs: *Training*

The main challenge is to calculate $\nabla \hat{R}(\mathbf{w})$

First, let's see what are the entries of $\mathbf{w}$: $\mathbf{w}$ contains *all weights and biases*.
Following our notations, we can say

$$\mathbf{w} = \left[\begin{array}{c} \mathbf{w}_1 [1, :] \\ \vdots \\ \mathbf{w}_1 [\mathcal{W}_1, :] \\ \vdots \\ \mathbf{w}_{L+1} [1, :] \\ \vdots \\ \mathbf{w}_{L+1} [\mathcal{W}_{L+1}, :] \end{array} \right] \begin{array}{l} \text{layer } \ell = 1 \\ \\ \\ \\ \text{layer } \ell = L + 1 \end{array} \longrightarrow \mathbf{w}_\ell [j, :] = \left[\begin{array}{c} w_\ell [j, 0] \\ w_\ell [j, 1] \\ \vdots \\ w_\ell [j, \mathcal{W}_{\ell-1}] \end{array} \right]$$

So, the entries of $\mathbf{w}$ are $w_\ell [j, i]$ for different choices of i, j and ℓ

Fully-Connected FNNs: *Training*

The main challenge is to calculate $\nabla \hat{R}(\mathbf{w})$

Let's try to open up the gradient: we need partial derivatives of $\hat{R}(\mathbf{w})$ with respect to $w_\ell [j, i]$ for $i = 0 : \mathcal{W}_{\ell-1}$, $j = 1 : \mathcal{W}_\ell$, and $\ell = 1 : L + 1$

$$\begin{aligned} \frac{\partial}{\partial w_\ell [j, i]} \hat{R}(\mathbf{w}) &= \frac{\partial}{\partial w_\ell [j, i]} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b) \\ &= \frac{1}{B} \sum_{b=1}^B \boxed{\frac{\partial}{\partial w_{i,j} [\ell]} \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b)} \end{aligned}$$

So, it's enough to develop an algorithm that computes partial derivative for a single **data-point**. Derivative of the risk is then average of these point-wise derivatives.

Fully-Connected FNNs: *Training*

Let's make an agreement: we consider a *data-point* $\mathbf{x}$ with *label* $\mathbf{v}$ and write

$$\frac{\partial}{\partial w_{\ell} [j, i]} \mathcal{L} (\text{PassF} (\mathbf{x} | \mathbf{w}), \mathbf{v}) = \frac{\partial}{\partial w_{\ell} [j, i]} \mathcal{L} (\mathbf{y}, \mathbf{v})$$

while keeping in mind that $\mathbf{y}$ is a function of $\mathbf{w}$

To determine the partial derivatives, we note that

$\mathbf{y}$ is a nested function of $\mathbf{w}$

so, we can determine the derivative via *chain rule*. Let's recall the *chain rule* and see how we can *apply it on a graph*

Review: Chain Rule

Assume $z = g(x)$ and $y = f(z)$: y is a *nested* function of x , as we can write

$$y = f(g(x))$$

Intuitively, we can say: if at point x we move with tiny step dx , z varies as

$$dz = \dot{g}(x)dx$$

This variation also varies y : moving from $z = g(x)$ with tiny step dz leads to

$$dy = \dot{f}(z)dz$$

So, we have

$$dy = \dot{f}(z)\dot{g}(x)dx$$

Review: Chain Rule

We have concluded that by moving x with dx , we get

$$dy = f'(z)g'(x)dx$$

On the other hand, we know that

$$dy = \frac{d}{dx} f(g(x))dx$$

This concludes the *chain rule*

Chain Rule: Scalar Form

The derivative of *nested* function $y = f(g(x))$ with respect to x is given by

$$\frac{dy}{dx} = \frac{d}{dx} f(g(x)) = f'(z)g'(x) = \frac{dy}{dz} \frac{dz}{dx}$$

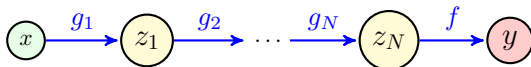
Computation Graph

We can extend this idea to deeper nested functions:

Let $z_1 = g_1(x)$ and $z_{n+1} = g_{n+1}(z_n)$ for $n = 1, \dots, N-1$; then, derivative of $y = f(z_N)$ with respect to x is given by

$$\frac{dy}{dx} = \frac{dy}{dz_N} \left(\prod_{n=1}^{N-1} \frac{dz_{n+1}}{dz_n} \right) \frac{dz_1}{dx} = \dot{f}(z_N) \left(\prod_{n=1}^{N-1} \dot{g}_{n+1}(z_n) \right) \dot{g}_1(x)$$

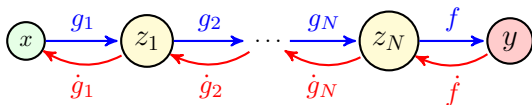
We can represent the chain rule, using a *computation graph*: for the deep nested function given above, the computation graph is given by



In this graph, we start from x and pass forward to $z_1 \rightarrow z_2 \rightarrow \dots$ until we get to y . In each pass, we determine next variable via the function on the link

Computation Graph

The derivative of y with respect to any variable on this graph is determined by a *backward pass* from y towards the variable



$$\frac{dy}{dz_N} = \dot{f}(z_N)$$

$$\frac{dy}{dz_{N-1}} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} = \dot{f}(z_N) \dot{g}_N(z_{N-1})$$

$$\vdots$$

$$\frac{dy}{dz_1} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} \cdots \frac{dz_2}{dz_1} = \dot{f}(z_N) \dot{g}_N(z_{N-1}) \cdots \dot{g}_2(z_2)$$

$$\frac{dy}{dx} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} \cdots \frac{dz_2}{dz_1} \frac{dz_1}{dx} = \dot{f}(z_N) \dot{g}_N(z_{N-1}) \cdots \dot{g}_2(z_2) \dot{g}_1(x)$$

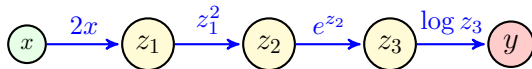
Computation Graph: Example

Example: y is a nested function of x through the following chain of functions:

$$z_1 = 2x \quad z_2 = z_1^2 \quad z_3 = e^{z_2} \quad y = \log z_3$$

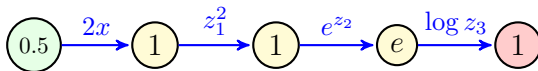
Determine the derivative of y with respect to x at $x = 0.5$.

Let's first plot the computation graph



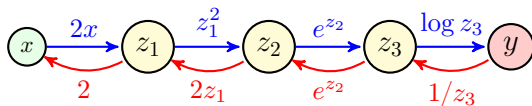
During the *forward pass* we get

$$z_1 = 2 \times 0.5 = 1 \rightarrow z_2 = 1^2 = 1 \rightarrow z_3 = e^1 = e \rightarrow y = \log e = 1$$

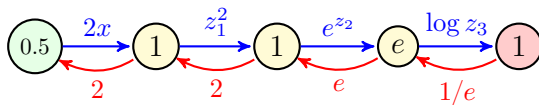


Computation Graph: *Example*

Now, we *pass backward* to determine the derivative

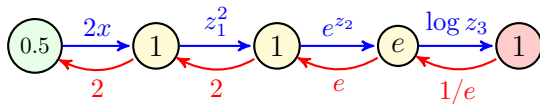


Let's first enter the values into the backward links



We now *navigate backward* to each variable that we want to determine the derivative y with respect to it

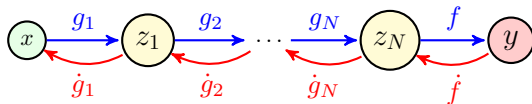
Computation Graph: *Example*



The derivatives are easily determined *recursively*

$$\begin{aligned}\frac{dy}{dz_3} &= \frac{1}{e} \\ \frac{dy}{dz_2} &= \frac{dy}{dz_3} \frac{dz_3}{dz_2} = \frac{e}{e} = 1 \\ \frac{dy}{dz_1} &= \frac{dy}{dz_2} \frac{dz_2}{dz_1} = 1 \times 2 = 2 \\ \frac{dy}{dx} &= \frac{dy}{dz_1} \frac{dz_1}{dx} = 2 \times 2 = 4\end{aligned}$$

Computation Graph



In the example, we had to *first compute the value of each variable*, in order to be able to compute the *values of the backward links*

This is an important fact that we should remember

Backward pass is only possible if we have already taken the forward pass

Review: Chain Rule

The nested function can be a multivariate: *assume for* $n = 1, \dots, N$

$$z_n = g_n(x)$$

and let the nested function be

$$y = f(z_1, \dots, z_N)$$

Let's follow the same logic: *starting from point* x , *we move with tiny step* dx

$$dz_n = \dot{g}_n(x) dx$$

These variations lead to variation dy *in the nested function*

$$dy = \nabla f(\mathbf{z})^\top d\mathbf{z} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} dz_n = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) dz_n$$

Review: Chain Rule

We can hence write

$$dy = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) dz_n = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x) dx$$

This concludes the *vector form* of the chain rule

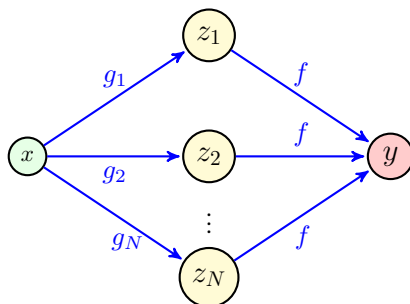
Chain Rule: Vector Form

Let $\mathbf{z} = [z_1, \dots, z_N]^T$ and $z_n = g_n(x)$. The derivative of nested function $y = f(\mathbf{z})$ with respect to x is given by

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x)$$

Computation Graph

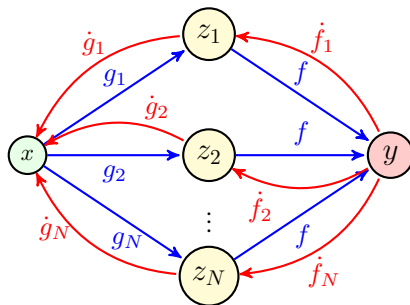
We can again represent the function via its vectorized *computation graph* and compute it by *passing forward* on this graph



We start from x and pass forward to $\mathbf{z} = [z_1, z_2, \dots, z_N]$. We then pass forward $\mathbf{z}$ to y . In each pass, we compute next variable via function on the link.

Computation Graph

The derivative with respect to any node is then given by *backward pass* towards the node on the computation graph

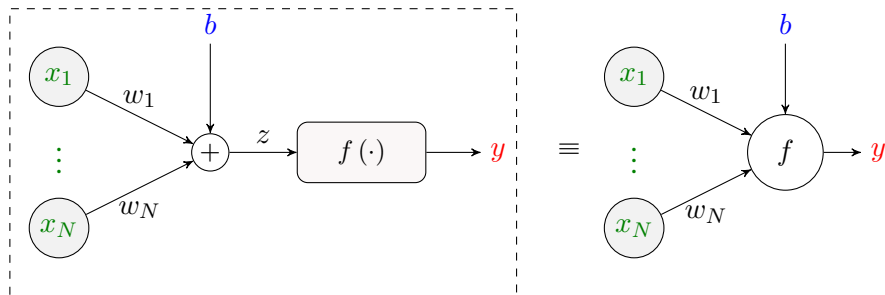


We add all backward passes towards x to determine the derivative

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x)$$

Computation Graph: *Single Neuron*

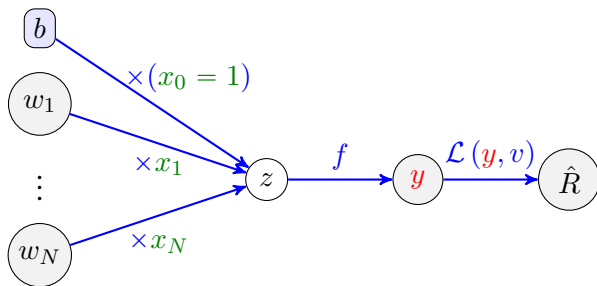
Let's now plot the *computation graph* of a single neuron and determine the gradient of the loss by *backward pass*



After passing the data-point $\mathbf{x}$ through the neuron, we get y and we calculate the loss for the *true label* v as $\mathcal{L}(y, v)$

Computation Graph: *Single Neuron*

The *computation graph* is hence given by

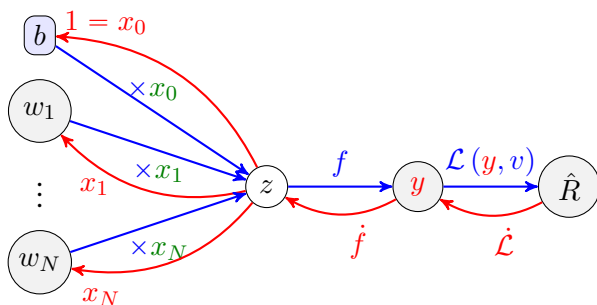


Here, the computation nodes are the *weights and bias of the neuron*

once we fix them, we can pass forward and get to the loss $\hat{R}$

Computation Graph: Single Neuron

Once passed forward, we can move backward to determine the derivatives

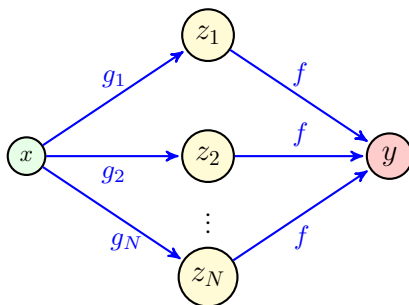


For a particular weight w_n , we can write (we drop arguments whenever clear)

$$\frac{\partial \hat{R}}{\partial w_n} = \frac{d\hat{R}}{dy} \frac{dy}{dz} \frac{\partial z}{\partial w_n} = \dot{\mathcal{L}} \dot{f} x_n$$

Computation Graph: *Multivariate Form*

Let's get back to the following *computation graph*



We define *vector-valued functions*, and show the graph *compactly*: let's define

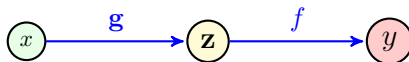
$$\mathbf{g}(x) = \begin{bmatrix} g_1(x) \\ \vdots \\ g_N(x) \end{bmatrix}$$

Computation Graph: *Multivariate Form*

Function $\mathbf{g}(\cdot)$ gets x as the input and returns all z_n 's in a **vector** $\mathbf{z}$, i.e.,

$$\mathbf{g}(x) = \begin{bmatrix} g_1(x) \\ \vdots \\ g_N(x) \end{bmatrix} = \begin{bmatrix} z_1 \\ \vdots \\ z_N \end{bmatrix} = \mathbf{z}$$

We now use this vectorized notation to simplify the *computation graph* as



The **forward pass** on this graph is exactly the same: we give x to the **vectorized function** $\mathbf{g}(\cdot)$ to get $\mathbf{z}$ which is then **passed forward** to $\mathbf{f}(\cdot)$ to get y

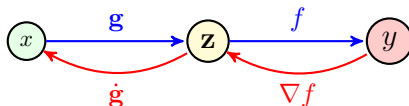
How does the **backward pass** look like then?

Computation Graph: *Multivariate Form*

We could define the derivative $\dot{\mathbf{g}}(\cdot)$ as the vector of derivatives $\dot{g}(\cdot)$

$$\dot{\mathbf{g}}(x) = \begin{bmatrix} \dot{g}_1(x) \\ \vdots \\ \dot{g}_N(x) \end{bmatrix} = \begin{bmatrix} \frac{dz_1}{dx} \\ \vdots \\ \frac{dz_N}{dx} \end{bmatrix} = \frac{d\mathbf{z}}{dx}$$

Let's show this **vectorized derivative** and **gradient** of f on the backward links

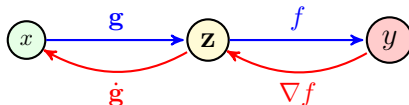


Well, we can **pass backward** as follows

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \nabla f(\mathbf{z})^T \dot{\mathbf{g}}(x)$$

Computation Graph: *Multivariate Form*

- + What can we conclude then?
- We can sketch the computation graph very compactly using *vectorized derivatives* and *gradients*
- + Does it mean that we should then *pass backward* exactly the same as in a computation graph with *scalar* variables and derivatives?
- Pretty much Yes! Only one delicate detail: we should *know how to multiply those gradients and vectorized derivatives!*



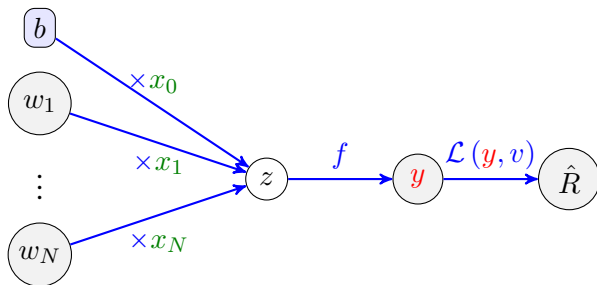
In our example, we determined *the inner product*

$$\frac{dy}{dx} = \nabla f^T \dot{g}$$

Computation Graph: *Multivariate Form*

- + How do we know *which type of product* we should use?
- Well! If you were in doubt, we could always do it by *expanding in terms of entries*; however, we are going to practice *all key functions* that appear in NN computation graphs!

Before we start with all key functions, let's get back to a *single neuron*



Computation Graph: *Multivariate Form*

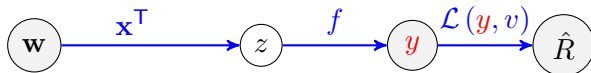
Let's define, as we did earlier, the following vectors

$$\mathbf{x} = \begin{bmatrix} x_0 = 1 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} w_0 = b \\ w_1 \\ \vdots \\ w_N \end{bmatrix}$$

Recall that **output** of the **neuron** is determined as $y = f(z)$ for

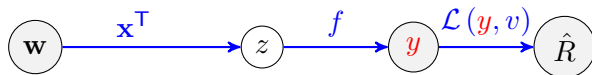
$$z = \mathbf{x}^T \mathbf{w}$$

So, we can show the **computation graph** compactly as



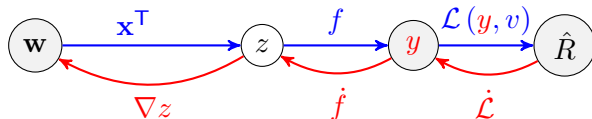
Computation Graph: *Multivariate Form*

Let's look at each link carefully: we pass *backward*, so we start with *last* link

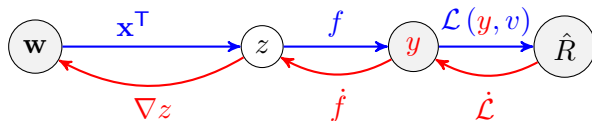


- $\hat{R}$ is a scalar function of scalar y , i.e., $\hat{R} = \mathcal{L}(y, v)$
 ↳ the backward link contains the *scalar derivative* $\dot{\mathcal{L}}$
- y is a scalar function of scalar z , i.e., $y = f(z)$
 ↳ the backward link contains the *scalar derivative* $\dot{f}$
- z is a scalar function of vector $\mathbf{w}$, i.e., $z = \mathbf{x}^T \mathbf{w}$
 ↳ the backward link contains the *gradient* ∇z

So, the graph with the *backward* links looks like



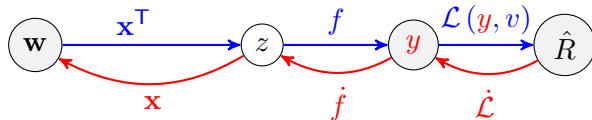
Computation Graph: Multivariate Form



We are almost complete; only we need to *calculate* ∇z

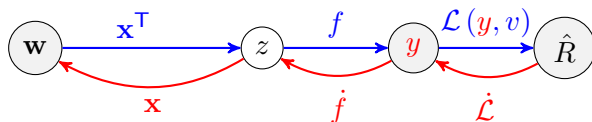
$$z = w_0 + w_1 x_1 + \dots + w_N x_N \rightsquigarrow \nabla z = \begin{bmatrix} \partial z / \partial w_0 \\ \partial z / \partial w_1 \\ \vdots \\ \partial z / \partial w_N \end{bmatrix} = \begin{bmatrix} 1 = x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} = \mathbf{x}$$

So, we are complete! Here is the *vectorized computation graph* of the *neuron*



Computation Graph: Multivariate Form

Now, how do we pass **backward** on this graph?



We arrived at y at the **end of forward pass**: at this point, we can determine

$$\frac{d\hat{R}}{dy} = \dot{\mathcal{L}}(y, v) = \dot{\mathcal{L}}$$

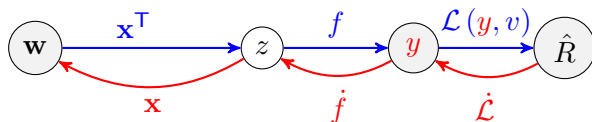
and we are at the **computing node** y . We then pass **backward** $\dot{\mathcal{L}}$. At node z , we can compute $\dot{f}(z)$, and use what we received from y to compute

$$\frac{d\hat{R}}{dz} = \frac{d\mathcal{L}}{dy} \frac{dy}{dz} = \dot{f} \dot{\mathcal{L}}$$

and pass it **backward**

Computation Graph: Multivariate Form

Now, how do we pass *backward* on this graph?



Arriving at w , we can determine $\nabla z = \mathbf{x}$ and use what we received from z to compute *what we want*

$$\nabla \hat{R} \text{ w.r.t. } \mathbf{w} \equiv \nabla_{\mathbf{w}} \hat{R} = \frac{d\hat{R}}{dz} \nabla z = \dot{f} \dot{\mathcal{L}} \mathbf{x}$$

- + Well! That seems easier!
- Right! Let's now try some **important cases**